11th IEEE Asia-Pacific Conference on Computer Science and Data Engineering (IEEE CSDE 2026) 1-3 November 2026, Cox's Bazar, Bangladesh

Robust Ransomware Detection Through Cross-Family Evaluation and Behavioral Feature Analysis

Gazi Mubtasim Tousif Nuhan[1], Md Alomgir Hossain[2], Md Sajjad Hossain[3],
Afsana Akter[4], Soyad Adnan Hossain Pranto[5], Shahi Pervez Pulok[6]
[1,2,3,4,5,6]Department of Computer Science Engineering, International University of Business Agriculture and Technology, Dhaka, Bangladesh
Email: [1]mubtasimtousif@gmail.com, [2]alomgir.hossain@iubat.edu, [3]sajjadhossaincse05@gmail.com,
[4]mim41839@gmail.com, [5]xhydraboost@gmail.com, [6]shahipervezpulok@gmail.com

***Abstract*—Machine-learning ransomware detectors are usually validated internally, so the test set contains the same families seen in training, as a review of 110 studies found. We evaluate seven classifiers on 15,638 Windows PE samples (10,876 benign and 4,762 ransomware from 12 families, drawn from a 21,752-sample corpus) using static PE-header and dynamic behavioral features. Six classifiers reach 98.1–99.6% accuracy on a standard internal split (Naive Bayes: 64.8%). Under a cross-family split and a 12-fold leave-one-family-out (LOFO) protocol, Gradient Boosting loses only 1.8 mean F1 points in LOFO, versus 15.8 for KNN and 32.0 for Naive Bayes; KNN and SVM lose mainly recall. A Random Forest feature ablation shows that static features do not generalize to unseen families (recall = 0.159), whereas dynamic features do (recall = 0.946). Voting and stacking ensembles do not outperform a single Gradient Boosting model (LOFO F1 0.9665 and 0.9727 vs. 0.9754; Stacking vs. Gradient Boosting Wilcoxon $p = 0.002$, median difference +0.0022). Permutation importance on held-out families highlights malicious-process and file-activity counts as the most transferable signals. All results come from a single dataset without packed or adversarial samples.**

***Keywords*—Ransomware, Machine Learning, Cross-Family Generalization, Feature Ablation, Ensemble Learning, Explainable AI, Cybersecurity**

I. Introduction

Ransomware remains one of the most damaging classes of cyberattack, and machine learning has become a standard reactive defense layered on top of preventative, zero-trust network design. A 2026 systematic review of 110 published ML-for-ransomware papers, spanning frameworks, surveys, and experiments, found consistently high reported accuracy (Random Forest: mean 94.52%, σ = 4.42; Meta-ensemble methods: mean 93.27%, σ = 7.14) but identified three structural weaknesses that limit how much those numbers actually tell us about real-world performance: (1) almost all surveyed experiments test internal validity only, i.e., the same ransomware families/variants appear in both the training and test partitions; (2) accuracy was the only metric reported consistently, while precision, recall, F1, and AUC were frequently omitted, hiding class-imbalance effects; and (3) despite the review's own recommendation for future work on multi-model (ensemble) solutions, no experiment in the surveyed set tested one. This paper is a follow-up, using the same data set, to fill in those three gaps. The following experiments all rely on a single labeled corpus (Section III-A) and this and other threats to validity are discussed explicitly in the Threats to Validity subsection of Section VI. The main contributions are:

- **Cross-family evaluation:** We test the ability to detect ransomware on unseen families of ransomware with the help of the cross-family and 12-fold LOFO protocols.
- **Comprehensive evaluation:** We evaluate seven classifiers using Accuracy, Precision, Recall, F1-score, Specificity and AUC.
- **Feature ablation:** We investigate the effect of feature ablation on static features and dynamic features to find out which type of feature is better at generalizing to unseen families.
- **Ensemble analysis:** We compare voting and stacking ensemble with classifiers in the cross-family and LOFO settings.
- **Explainability:** We use permutation importance on held-out families to identify the most important transferable features
- **Category-level validation:** We run five-way classification to ensure the capability to detect ransomware.

The related work is presented in Section II. The data set, data pre-processing, models and protocols are described in Section III. The experimental set up is provided on Section IV. Section V reports results. Findings and threats to validity are discussed in Section VI. Section VII concludes

II. Related Work

Veach and Abualkibash [1] surveyed 110 papers (2019–2026) on ML/AI ransomware detection: Random Forest was most tested (39 appearances, 94.52% mean accuracy, σ=4.42%), followed by meta-ensembles (29, 93.27%, σ=7.14%), Decision Tree (22, 93.68%), Neural Networks (20, 90.17%), Bayesian methods (20, 88.63%), KNN (20, 91.81%), SVM (18, 90.00%), and Logistic Regression (12, 88.78%, σ=10.44%). The review names three motivating gaps: no cross-family testing, inconsistent F1/precision/recall reporting,

979-8-3195-4477-3/26/$31.00 © 2026 IEEE

and unexplored multi-model solutions. Subsequent surveys corroborate these: Ispahany et al. [2] and Alraizza and Algarni [3] report only internal-validation figures; Ferdous et al. [4] and Al-Rahhal et al. [5] extend to static/dynamic/hybrid taxonomies without an empirical cross-family comparison; Kritika [6], screening 23 studies (2017–2024), likewise finds accuracy the dominant, often sole, reported metric.

Cross-family generalization has been explored more broadly in malware classification than in ransomware specifically. DeepOrigin [7] reports a sharp accuracy drop on novel families. ISAnWin [8] uses Siamese zero-shot learning to reach 82% accuracy on unseen variants but does not decompose this into precision/recall/F1. MalFSLDF [9] and FARM [10] apply few-shot/metric-learning to novel families; FARM's mean F1 of 0.85 on its own held-out split is broadly comparable to – though not directly commensurate with, given the different dataset/protocol – the LOFO results in Section V-C. MOTIF [11] benchmarks novel-family detection with LightGBM and MalConv2; MalwareDNA [12] unifies detection, family classification, and novel-family identification in one semi-supervised model.

On static-vs-dynamic, a SHAP-based study [13] notes PE-header attributes encode build-time metadata rather than runtime behavior, causing family overlap in static feature space. A transfer-learning approach [14] using dynamic sandbox features reports static-only methods plateau near 93.3% accuracy, and a structural static study [15] combining PE headers, DLLs, and opcodes remains vulnerable to packer-artifact overfitting. Only 1 of the 110 papers surveyed by [1] used explainable AI: Rani and Dhavale [16] apply SHAP to API-call-based family classification, and Abraham et al. [17] similarly find behavioral/process-level features dominate importance – though both score on the training distribution rather than held-out families.

Closest to our static-feature setting, Moreira et al. [18] classify ransomware from portable-executable header data using an Xception convolutional neural network.

III. Methodology

A. Data Description

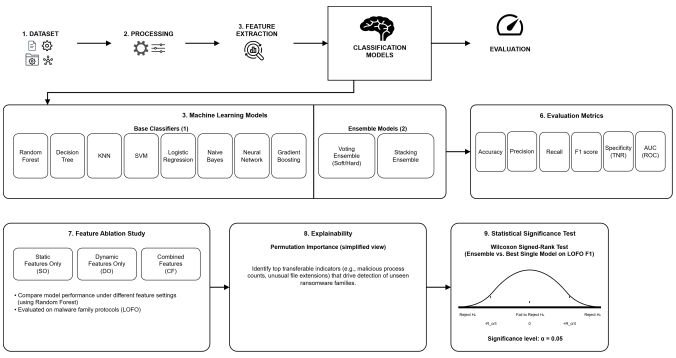


Fig. 1: Overall Methodology and Evaluation Framework.

The dataset is drawn from the publicly available Ransomware Dataset 2024 [19], which contains 21,752 labeled Windows PE samples with three label granularities: binary Class (10,876 benign / 10,876 malware), five-way Category (Benign 10,876; Ransomware 4,762; RAT 2,647; Stealer 2,018; Trojan 1,449), and a 27-value Family label. Of these, 12 families are ransomware, ranging from Phobos (n=550) down to Ragnar (n=133): Phobos, Dharma, Shade, WannaCry, Ryuk, LockBit, Gandcrab, REvil, DarkSide, Cerber, Maze, and Ragnar. Each sample carries 77 raw columns spanning static PE-header attributes and dynamic behavioral counters from sandboxed execution. All ransomware-detection experiments (Sections V-A to V-G) use the 15,638 benign and ransomware samples (10,876 benign, 4,762 ransomware; 30.5% minority class). The RAT, Stealer, and Trojan samples are used only in the five-way Category test (Section V-H).

B. Data Preprocessing

Raw columns were converted to numeric form in three steps: (1) hexadecimal PE-header fields (e.g., EntryPoint, SizeOfImage) parsed to integers; (2) flag-list columns (e.g., DllCharacteristics) reduced to a count of active flags; (3) categorical fields (file_extension, PEType, MachineType, Subsystem) one-hot encoded. The 77 raw columns thus become 83 features, split into **static** (65: PE-header/structural) and **dynamic** (18: registry, network, process, file, DLL/API activity). Apart from these conversions, no features were hand-engineered or dropped, so the static-vs-dynamic comparison (Section V-E) is not confounded by unequal engineering effort. Missing values were imputed with the training-fold median (Section V-D), and features were standardized within each split to avoid leakage:

$$x' = \frac{x - \mu_{\text{train}}}{\sigma_{\text{train}}}. \tag{1}$$

Permutation importance was used afterward, on held-out data, to identify which features actually drove predictions (Section V-G).

C. Classification Models

Seven base classifiers spanning the families most common in the surveyed literature were evaluated, each summarized briefly below.

Decision Tree (DT) recursively partitions the feature space by minimizing Gini impurity at each split,

$$G = 1 - \sum_{c} p_c^2 \tag{2}$$

where p_c is the proportion of class c samples in a node.

Random Forest (RF) trains 200 such trees on bootstrap samples with random feature subsets and predicts by majority vote, reducing the variance of any single tree.

k-Nearest Neighbors (KNN, $k = 5$) classifies a sample by the majority label among its k closest training points under Euclidean distance,

$$d(x, x') = \|x - x'\|_2. \tag{3}$$

Support Vector Machine (SVM) finds the maximum-margin decision boundary $f(x) = \text{sign}(w^\top \phi(x) + b)$ using an

RBF kernel, $K(x, x') = \exp(-\gamma\|x - x'\|^2)$, with probability estimates enabled for ROC/AUC scoring.

Logistic Regression (LR) models the ransomware probability directly as

$$P(y = 1 \mid x) = \frac{1}{1 + e^{-(w^\top x + b)}}. \tag{4}$$

Gaussian Naive Bayes (NB) applies Bayes' rule under a feature-independence assumption,

$$P(y \mid x) \propto P(y) \prod_i P(x_i \mid y), \tag{5}$$

with each $P(x_i \mid y)$ modeled as Gaussian.

Gradient Boosting (GB), representing the "meta-algorithm" ensemble family, builds an additive model stage-wise,

$$F_m(x) = F_{m-1}(x) + \eta\, h_m(x). \tag{6}$$

IV. Experimental Setup

A. Software, Hardware, and Reproducibility

All experiments used Python 3.12 (scikit-learn, pandas/NumPy, SciPy for the Wilcoxon test, Matplotlib), with a fixed seed (RANDOM_STATE = 42) across NumPy and all scikit-learn estimators so every result in Section V is exactly reproducible.

B. Hyperparameter Configuration

Table I lists the configuration for every model in Section III-C plus shared data-handling/evaluation settings; Section V-D reports the grid-search ranges used to test whether defaults understate SVM/KNN/RF performance.

TABLE I: Hyperparameters and Experimental Configuration

#	Parameter	Value
1	Random Forest	200 trees, default depth
2	Decision Tree	Default (Gini, unlimited depth)
3	k-NN	$k = 5$, Euclidean distance
4	SVM kernel	RBF, probability=True
5	Logistic Regression	max_iter = 1000
6	Gradient Boosting	100 estimators, $\eta = 0.1$
7	Voting Ensemble	Soft voting (RF+DT+GB)
8	Stacking Meta-Learner	Logistic Regression, 3-fold CV
9	Train/Test (Internal)	70% / 30%, stratified
10	Cross-Family Split	8 train / 4 test families
11	LOFO Folds	12 (one family held out/fold)
12	Missing-Value Imputation	Median (train-fold only)
13	Feature Scaling	Z-score standardization
14	Random Seed	42 (fixed, all libraries)
15	Core Metrics	Accuracy, Precision, Recall, F1
16	Additional Metrics	Specificity, AUC, Confusion Matrix

V. Results

A. Internal Validity Baseline

Table II reports all six metrics for the internal 70/30 split. Six of the seven classifiers reach 98.08–99.62% accuracy, matching or exceeding the 94–99% range reported in the reviewed literature: RF reaches 99.62%/0.9937 F1 and GB 99.57%/0.9930 F1. Naive Bayes is the outlier (64.75%, 0.6323 F1), consistent with its independence assumption fitting correlated features poorly. Fig. 3shows RF and GB near the top-left corner (AUC = 1.000 and 0.9999); DT, SVM, KNN, and LR cluster at AUC 0.993–0.999; Naive Bayes lags (AUC = 0.749), achieving high recall only by over-flagging benign files (Specificity = 0.495). Bootstrap 95% CIs (1000 resamples) confirm these rankings hold beyond a single split, with RF/GB intervals overlapping each other but not KNN or Naive Bayes.

TABLE II: Internal-Validity Results (Same-Family Random Split)

Classifier	Acc.	Prec.	Rec.	F1	Spec.	AUC
Random Forest	**.9962**	**.9937**	.9937	**.9937**	**.9972**	**1.000**
Decision Tree	.9940	.9868	.9937	.9902	.9942	.9939
KNN	.9808	.9718	.9650	.9684	.9877	.9934
SVM	.9908	.9839	.9860	.9850	.9930	.9990
Logistic Regression	.9934	.9868	.9916	.9892	.9942	.9989
Naive Bayes	.6475	.4633	.9951	.6323	.4952	.7491
Gradient Boosting	.9957	.9889	**.9972**	.9930	.9951	.9999

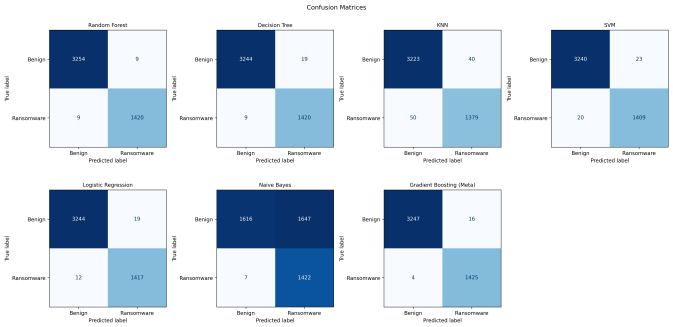


Fig. 2: Confusion Matrices, Internal-Validity Split.

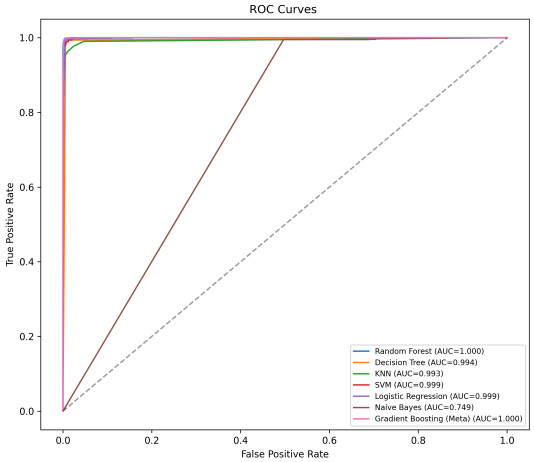


Fig. 3: ROC Curves, Internal-Validity Split.

B. Cross-Family Generalization

Table III is a repeat of the evaluation from the 8-vs-4 family split. If only accuracy is considered, little changed is suggested (RF 98.45%, GB 99.47%), but Table IV tells a different

story: KNN loses 0.1225 F1 (0.9684→0.8459) and SVM loses 0.0950 (0.9850→0.8900), driven by collapsing recall (KNN 0.965→0.753; SVM 0.986→0.813) – margin/distance-based models maintain a high level of precision and more often fail to detect unseen-family ransomware. Tree-ensemble methods are far more stable, with GB losing only 0.0026 F1 and DT 0.0083. F1 values are reported to four decimal places throughout, agreeing with Table V.

TABLE III: Cross-Family Generalization (Unseen: LockBit, REvil, DarkSide, Ragnar)

Classifier	Acc.	Prec.	Rec.	F1	Spec.	AUC
Random Forest	.9845	.9933	.9505	.9715	**.9975**	**.9996**
Decision Tree	.9900	.9887	.9753	.9819	.9957	.9855
KNN	.9238	.9642	.7534	.8459	.9893	.9386
SVM	.9442	.9826	.8132	.8900	.9945	.9964
Logistic Regression	.9821	.9867	.9481	.9670	.9951	.9985
Naive Bayes	.6433	.4358	.9697	.6013	.5179	.7502
Gradient Boosting	**.9947**	**.9936**	**.9872**	**.9904**	.9975	**.9996**

TABLE IV: Generalization Gap: Internal vs. Cross-Family (Ranked by F1 Drop)

Classifier	Acc. Drop	F1 (Int. → Cross)	F1 Drop
KNN	.0570	.9684 → .8459	.1225
SVM	.0466	.9850 → .8900	.0950
Naive Bayes	.0042	.6323 → .6013	.0310
Random Forest	.0117	.9937 → .9715	.0222
Logistic Regression	.0113	.9892 → .9670	.0222
Decision Tree	.0040	.9902 → .9819	.0083
Gradient Boosting	**.0010**	.9930 → .9904	**.0026**

C. Robustness Check: 12-Fold Leave-One-Family-Out

A single 8-vs-4 split depends on which four families land in the test set, so we also ran a 12-fold leave-one-family-out (LOFO) protocol that holds out each ransomware family once (12 folds × 7 classifiers = 84 runs). Table V confirms the trend from Section V-B. Naive Bayes' mean F1 falls to 0.3120 (σ=0.0801) and KNN's to 0.8107 (σ=0.1434). Gradient Boosting is the most stable (0.9754, σ=0.0314, only 0.0176 below internal), followed by Random Forest (0.9614) and Decision Tree (0.9546).

TABLE V: LOFO (12 Folds): Mean F1 ± Std, Drop from Internal F1

Classifier	Internal F1	LOFO Mean F1 ($\pm\sigma$)	Drop
Naive Bayes	.6323	.3120 ± .0801	.3203
KNN	.9684	.8107 ± .1434	.1577
Logistic Regression	.9892	.9172 ± .1236	.0720
SVM	.9850	.9210 ± .0517	.0640
Random Forest	**.9937**	.9614 ± .0717	.0323
Decision Tree	.9902	.9546 ± .0391	.0356
Gradient Boosting	.9930	**.9754 ± .0314**	**.0176**

Table VI tests whether small families alone drive the drop. KNN and Naive Bayes do worse on the four smallest families (KNN mean F1 0.686 vs. 0.873 on the other eight; Naive Bayes 0.224 vs. 0.356), but size is not the whole story: KNN's weakest folds are Maze (n=290, F1 0.477) and Shade (n=504, 0.646). Gradient Boosting stays within 0.879–0.991 regardless of size (Ragnar, n=133: 0.9597; Phobos, n=550: 0.9910), with

TABLE VI: Per-Family F1 on LOFO Folds (Sorted by Family Size)

Held-out Family	N	Naive Bayes	KNN	Grad. Boost.
Ragnar	133	.1385	.7354	.9597
Maze	290	.2673	.4773	.9864
Cerber	332	.2096	.7300	.9821
DarkSide	335	.2810	.8000	.9867
REvil	347	.2896	.8746	.9843
Gandcrab	382	.3213	.8230	.9857
LockBit	438	.3528	.9324	**.9910**
Ryuk	449	.3235	.8977	.9878
WannaCry	497	.3832	.9149	.9851
Shade	504	.3864	.6460	.8790
Dharma	505	.3844	.9457	.9862
Phobos	550	**.4061**	**.9512**	**.9910**

Shade (0.879) as its only clear low fold. The gap is therefore not purely a small-sample artifact.

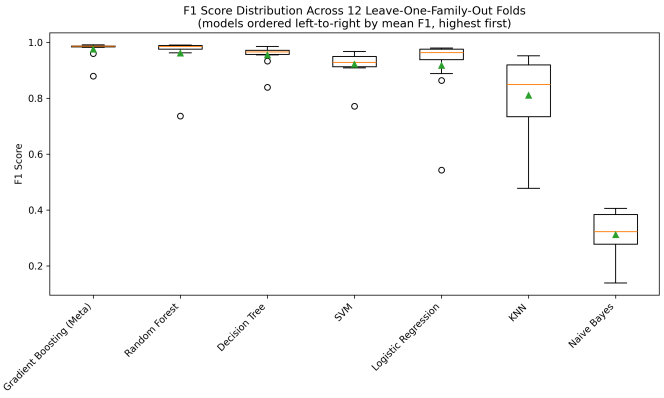


Fig. 4: F1 distribution across the 12 LOFO folds for all classifiers. Classifiers are ordered left to right by descending mean F1. Points beyond the box whiskers are individual held-out-family outlier folds. (cf. Table VI).

D. Robustness to Hyperparameters

Tuning. SVM, KNN, and Random Forest were grid-searched on a training-only validation fold (held-out families untouched) and re-evaluated on the cross-family test set: SVM over $C \in \{0.1, 1, 10, 100\}$, $\gamma \in \{0.001, 0.01, 0.1, \text{scale}\}$; KNN over $k \in \{3, 5, 7, 9, 11, 15\}$ with Euclidean/Manhattan distance; RF over $\{100, 200, 400\}$ trees and max depth $\in \{\text{None}, 10, 20\}$. Table VII shows tuning helps SVM (C=10, γ=0.001), lifting recall from 0.813 to 0.941 and closing about three-quarters of its F1 gap, while KNN and RF do not improve. Gradient Boosting, Decision Tree, and Logistic Regression used default settings, and SVM's best γ lies at the grid boundary, so a wider search might help it further.

TABLE VII: Effect of Hyperparameter Tuning on Cross-Family F1

Model	Default F1	Tuned F1	Improvement
SVM	.8900	**.9628**	**+.0728**
KNN	.8459	.8449	−.0010
Random Forest	.9715	.9715	.0000

Class imbalance. Stratified splitting prevents leakage but not training imbalance, so we also tested `class_weight='balanced'` and SMOTE (training fold only). Both improved minority-class recall: LR F1 rose 0.967→0.982 and SVM recall 0.813→0.859 (both balanced), GB peaked under SMOTE (F1 0.9924 vs. 0.9904), and RF was essentially unchanged. Imbalance thus mainly hurts weaker classifiers' recall. Naive Bayes was not rebalanced, so its poor performance is attributed mainly to its independence assumption.

Imputation. Re-running the RF cross-family experiment with mean and KNN (k=5) imputation instead of median gives F1 of 0.9719 and 0.9715 vs. 0.9715 for median – a spread of only 0.0004, supporting imputation choice as a robustness non-issue.

E. Feature Ablation: Static vs. Dynamic vs. Combined

This is the headline experiment: RF is retrained on the same cross-family split using only static features, only dynamic features, or both (Table VIII). Static-only features collapse: recall falls to 0.1588 (F1=0.2711) despite high precision (0.9256) – the model essentially stops flagging unseen-family ransomware, consistent with static PE-header features encoding family-specific packer/compiler artifacts rather than general ransomware behavior. Dynamic-only features are far more robust (recall 0.9457, F1 0.9673), and combining both gives only a marginal further gain (F1 0.9715) – static features add little once dynamic behavioral features are present, and would actively mislead a static-only model.

TABLE VIII: Feature Ablation, Cross-Family Split (Random Forest)

Feature Set	Acc.	Prec.	Rec.	F1	Spec.	AUC
Static-only	.7631	.9256	.1588	.2711	.9951	.9167
Dynamic-only	.9823	.9900	.9457	.9673	.9963	.9993
Combined	**.9845**	**.9933**	**.9505**	**.9715**	**.9975**	**.9996**

F. Ensemble Robustness

Table IX tests the reference review's call for a multi-model solution. On the single 8-vs-4 split, Stacking eliminates the drop (0.9968 → 0.9969) and Voting narrows it (0.0038); but under the stricter 12-fold LOFO test, Stacking (0.9727, $\sigma = 0.0342$) and Voting (0.9665, $\sigma = 0.0344$) sit slightly below single-model GB (0.9754, $\sigma = 0.0314$). A paired Wilcoxon test confirms Gradient Boosting holds a small but significant edge over Stacking (+0.0022 median F1 difference, $p = 0.0020$, $n = 12$). Across all nine models, Gradient Boosting significantly outperforms every weaker single model and both ensembles (Voting, Stacking) and is statistically indistinguishable only from Random Forest, while also showing the lowest fold-to-fold instability of any model ($\sigma = 0.0314$ vs. KNN's 0.1434). Under LOFO, neither ensemble improved on a single Gradient Boosting model, which was run with default settings Table I.

TABLE IX: Ensemble Generalization Gap (Single 8-vs-4 Split)

Model	Internal Acc.	Cross-Family Acc.	Drop
Voting Ensemble	.9960	.9922	.0038
Stacking Ensemble	**.9968**	**.9969**	**−.0001**

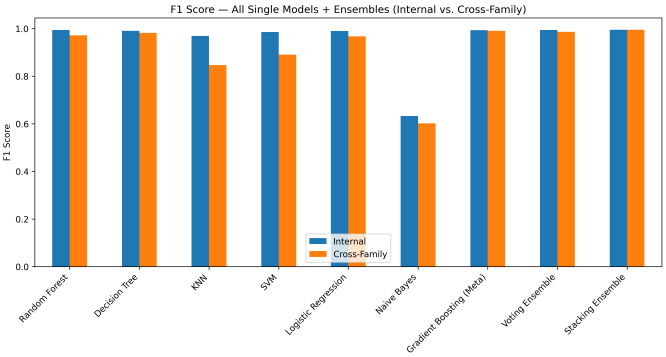


Fig. 5: F1 score, single models vs. ensembles (internal vs. cross-family).

G. Explainability

Only 1 of the 110 reviewed papers used explainable AI. Table X reports top features by permutation importance, computed directly on held-out families rather than the training distribution – a more meaningful measure of what drives generalization, since it is scored on data the model never trained on. `processes_malicious` (dynamic) dominates by a wide margin, consistent with Section V-E's finding that dynamic features drive generalization; static PE-header fields appear only far down the ranking. This also gives the finer-grained, feature-level breakdown within the dynamic group, beyond the group-level comparison in Table VIII.

TABLE X: Top Features Driving Cross-Family Detection (Permutation Importance, Held-Out Families)

Feature	Score
processes_malicious	.1945
files_suspicious	.0154
files_unknown	.0110
files_text	.0069
Subsystem_WINDOWS_GUI	.0032
address_of_ne_header	.0022
processes_monitored	.0012

H. Category-Level Bonus Test

This test uses a stratified 70/30 split on the 5-way Category label (same 83-feature set, 200-tree RF, median-imputation pipeline, and RANDOM_STATE=42 as the binary experiments) – 15,226 training / 6,526 test samples, chosen to test category-level discriminability rather than cross-family generalization. The five-way RF classifier reaches 98% overall accuracy, 0.98 weighted F1; Ransomware is detected with 0.98 precision / 0.99 recall, as reliably as in the binary task, while RAT is hardest to separate (0.90 precision, 0.96 recall) – indicating the model learns ransomware-specific signal rather than a generic malware/benign boundary.

VI. Discussion

Three insights emerge from the findings. First, accuracy hides generalization failure: all seven classifiers lose accuracy from internal to cross-family testing, and the largest losses are in ransomware recall and F1 (Tables III–IV). Accuracy-only reporting, as in most of the 110 reviewed papers, is therefore a weak indicator of performance on unseen families. Second, the static-vs-dynamic ablation indicates that static PE-header features encode family-specific artifacts, whereas behavioral features transfer to unseen families, which favors behavior-based over signature-based detection. Third, ensembling is not a panacea: Stacking appears to beat Gradient Boosting on the single 8-vs-4 split (accuracy 0.9969 vs. 0.9947), but under LOFO both ensembles fall slightly below it, a difference the single split would not reveal.

Threats to Validity

Results come from a single Kaggle-sourced corpus [19] with no independent cross-dataset validation (e.g., VirusShare, Hybrid-Analysis) – the study's main limitation and follow-up priority. No cross-dataset validation (e.g., VirusShare, Kaggle) – the study's main limitation and follow-up priority. Family sizes are also imbalanced (Phobos n=550 vs. Ragnar n=133), though Table VI shows this does not explain the entire LOFO gap.

No evaluation was performed against packed, obfuscated, fileless, or living-off-the-land ransomware; robustness claims apply only to the unobfuscated samples here and should not be expected to hold against an adaptive adversary.

Dynamic features also carry a real deployment cost: sandbox detonation takes seconds to minutes versus sub-millisecond static parsing, and sandbox-evasion techniques (not modeled here) could make an evasive sample appear benign. Held-out permutation importance (Table X) partly mitigates this by identifying which dynamic sub-features matter most, though a full per-feature ablation was not done. Comparisons to prior work in Section II remain illustrative of reporting practice rather than a strict cross-dataset benchmark.

VII. Conclusion and Future Work

This paper tested the three gaps identified by a systematic review of 110 studies: internal-only validation, inconsistent metric reporting, and untested multi-model solutions. Under cross-family and leave-one-family-out evaluation on a single dataset, tree-based classifiers generalized far better than KNN, SVM, and Naive Bayes (mean LOFO F1 drop of 0.0176 for Gradient Boosting vs. 0.1577 for KNN and 0.3203 for Naive Bayes). Dynamic behavioral features drove this generalization: static-only features reached an F1 of only 0.2711 on unseen families, compared with 0.9673 for dynamic-only features. Voting and stacking ensembles did not improve on a single Gradient Boosting model. Future work should validate on independent public datasets, test packed, obfuscated, and evasive samples, measure the real-time overhead of the dynamic-feature pipeline, and consider the human-factors dimension noted in the reference review.

References

[1] A. Veach and M. Abualkibash, "Analyzing machine learning techniques in detecting and preventing ransomware," *International Journal of Computing and Digital Systems*, vol. 19, no. 1, 2026.

[2] J. Ispahany, M. R. Islam, M. Z. Islam, and M. A. Khan, "Ransomware detection using machine learning: A review, research limitations and future directions," *IEEE Access*, vol. 12, pp. 68785–68813, 2024.

[3] A. Alraizza and A. Algarni, "Ransomware detection using machine learning: A survey," *Big Data and Cognitive Computing*, vol. 7, no. 3, p. 143, 2023.

[4] J. Ferdous, R. Islam, A. Mahboubi, and M. Z. Islam, "AI-based ransomware detection: A comprehensive review," *IEEE Access*, vol. 12, pp. 136666–136695, 2024.

[5] M. Al-Rahhal, S. Khawatmi, F. Farha, and M. M. Khatib, "A comprehensive survey of machine learning-based ransomware detection: Taxonomy, behavioral analysis, and emerging defense paradigms," *International Journal of Engineering Research & Technology (IJERT)*, vol. 15, no. 6, 2026.

[6] Kritika, "A comprehensive literature review on ransomware detection using deep learning," *Cyber Security and Applications*, vol. 3, p. 100078, 2025.

[7] I. Cordonsky, I. Rosenberg, G. Sicard, and E. O. David, "DeepOrigin: End-to-end deep learning for detection of new malware families," in *2018 Int. Joint Conf. on Neural Networks (IJCNN)*, 2018, pp. 1–7.

[8] U. Tayyab, F. B. Khan, A. Khan, M. H. Durad, F. A. Khan, and A. Ali, "ISAnWin: Inductive generalized zero-shot learning using deep CNN for malware detection across Windows and Android platforms," *PeerJ Computer Science*, vol. 10, p. e2604, 2024.

[9] W. Guo, J. Xue, Y. Lin, W. Du, J. Hu, N. Shi, and W. Han, "MalFSLDF: A few-shot learning-based malware family detection framework," *International Journal of Intelligent Systems*, vol. 2025, no. 1, p. 7390905, 2025.

[10] N. H. Guldemir, O. Olukoya, and J. Martínez-del-Rincón, "FARM: Few-shot adaptive malware family classification under concept drift," 2026, arXiv:2601.17907.

[11] R. J. Joyce, D. Amlani, C. Nicholas, and E. Raff, "MOTIF: A large malware reference dataset with ground truth family labels," 2021, arXiv:2111.15031.

[12] M. E. Eren, M. Bhattarai, K. Rasmussen, B. S. Alexandrov, and C. Nicholas, "MalwareDNA: Simultaneous classification of malware, malware families, and novel malware," in *2023 IEEE Int. Conf. on Intelligence and Security Informatics (ISI)*, 2023, pp. 1–3.

[13] G. Barnes and A. Ghafarian, "Machine learning-based static ransomware detection using PE header features and SHAP interpretation," *Journal of Cybersecurity and Privacy*, vol. 6, no. 2, p. 58, 2026.

[14] J. Ferdous, R. Islam, A. Mahboubi, and M. Z. Islam, "A novel technique for ransomware detection using image based dynamic features and transfer learning to address dataset limitations," *Scientific Reports*, vol. 15, no. 1, p. 32342, 2025.

[15] C. C. Moreira, D. C. Moreira, and C. de S. de Sales Jr., "A comprehensive analysis combining structural features for detection of new ransomware families," *Journal of Information Security and Applications*, vol. 81, p. 103716, 2024.

[16] N. Rani and S. V. Dhavale, "Application of explainable machine learning in detecting and classifying ransomware families based on API call analysis," 2022, arXiv:2210.11235.

[17] B. Abraham, R. Kumar, S. Shukla, and A. Srivastava, "Explainable machine learning for ransomware detection using SHAP," *Electronics*, vol. 11, no. 19, p. 3079, 2022.

[18] C. C. Moreira, D. C. Moreira, and C. de S. de Sales Jr., "Improving ransomware detection based on portable executable header using Xception convolutional neural network," *Computers & Security*, vol. 130, p. 103265, 2023.

[19] Mexwell, "Ransomware Dataset 2024," Kaggle, 2024. [Online]. Available: https://www.kaggle.com/datasets/mexwell/ransomware-dataset-2024